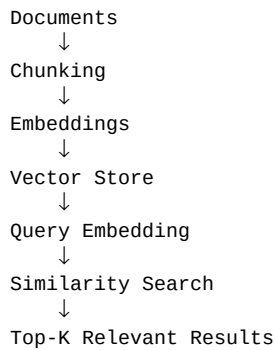


Semantic Search with Embeddings and Vector Stores

A Python-based semantic document search project that converts documents and queries into embeddings, stores the vectors in a vector store, and retrieves the most relevant document chunks using similarity search.

Project Overview

The project demonstrates a complete semantic search pipeline:



The project supports multiple embedding and vector-store backends, making it possible to compare different approaches.

Project Structure

```
semantic_search/
├──
├── chunking.py
├── embedder.py
├── search_engine.py
├── vector_store_faiss.py
├── vector_store_chroma.py
├── vector_store_numpy.py
├── demo.py
└── requirements.txt
```

chunking.py

Splits documents into smaller chunks before embedding. The chunking process works at the sentence level and packs sentences into approximately fixed-size word windows. It also supports overlap between consecutive chunks.

Default settings: Chunk size: 400 words; Overlap: 60 words.

Each chunk contains information such as Chunk ID, Document ID, Document title, Chunk index, and Metadata.

embedder.py

Provides different embedding implementations through a common interface.

Sentence Transformer

The default semantic embedding model is all-MiniLM-L6-v2. It produces 384-dimensional embeddings and runs locally.

OpenAI Embeddings

An OpenAI embedding backend is also available when the OpenAI package, API key, and network access are configured.

TF-IDF

A lightweight TF-IDF implementation is provided as a fallback. It does not produce deep-learning semantic embeddings, but it allows the complete search pipeline to run without heavy ML dependencies or internet access.

The auto backend first tries Sentence Transformers and falls back to TF-IDF if necessary.

search_engine.py

This is the main component that connects chunking, embeddings, and vector stores. The DocumentSearchEngine supports faiss, chroma, and numpy.

During ingestion:

```
Documents
↓
Chunk documents
↓
Create embeddings
↓
Create vector store
↓
Store vectors and chunks
```

During search:

```
User query
↓
Create query embedding
↓
Search vector store
↓
Return top-K results
```

Vector Stores

FAISS

FAISS uses normalized embeddings with inner-product search, which corresponds to cosine similarity. It also supports saving and loading the index from disk.

```
python demo.py --vector faiss
```

ChromaDB

ChromaDB stores embeddings, documents, and metadata. The implementation uses cosine similarity and supports optional persistent storage.

```
python demo.py --vector chroma
```

NumPy

The NumPy implementation provides a simple in-memory vector store. It calculates similarity using the dot product of normalized vectors.

```
python demo.py --vector numpy
```

Demo

demo.py contains five sample documents: Remote Work Policy, Expense Reimbursement Guide, Onboarding Checklist for New Hires, Data Retention and Privacy Policy, and Vector Database Overview.

The demo indexes the documents and runs sample queries such as:

```
How many days can I work from home?
What's the daily limit for meal expenses on a business trip?
What happens to customer data after they close their account?
```

```
How does similarity search work for embeddings?  
What does a new employee do in their first week?
```

The demo returns the top matching document chunks with similarity scores.

Installation

Create and activate a Python environment, then install the dependencies:

```
pip install -r requirements.txt
```

The project requires NumPy and Sentence Transformers. FAISS and ChromaDB are included for their respective vector-store backends.

Running the Demo

FAISS

```
python demo.py --vector faiss
```

ChromaDB

```
python demo.py --vector chroma
```

NumPy

```
python demo.py --vector numpy
```

You can also select the embedding backend:

```
python demo.py --vector faiss --embedder sentence-transformers  
python demo.py --vector faiss --embedder tfidf
```

The number of returned results can be changed with `--top_k`:

```
python demo.py --vector faiss --top_k 5
```

Example Output

For the query: How many days can I work from home?

```
1. [0.637] Remote Work Policy  
   Employees may work from home up to three days per week...  
  
2. [0.357] Onboarding Checklist for New Hires  
   ...  
  
3. [0.202] Data Retention and Privacy Policy  
   ...
```

The score represents the similarity between the query embedding and the stored document embedding. Higher scores indicate more relevant results.

Requirements

```
numpy  
sentence-transformers  
faiss-cpu  
chromadb
```

The OpenAI package is optional and is only required when using the OpenAI embedding backend.

Key Features

- Sentence-based document chunking
- Chunk overlap

- Local semantic embeddings
- Sentence Transformers support
- OpenAI embedding support
- TF-IDF fallback
- FAISS vector search
- ChromaDB vector search
- NumPy vector search
- Top-K similarity retrieval
- Metadata support
- FAISS index persistence

Overall Workflow

1. Load documents
↓
2. Split documents into chunks
↓
3. Generate embeddings
↓
4. Store embeddings in a vector store
↓
5. Convert user query into an embedding
↓
6. Compare query with stored vectors
↓
7. Retrieve the most relevant chunks
↓
8. Display document title, score, and content

Purpose

This project demonstrates the core components behind semantic search and retrieval systems. It provides a modular structure where the embedding method and vector-store implementation can be changed without changing the complete search workflow.